



Linked List - Class 3

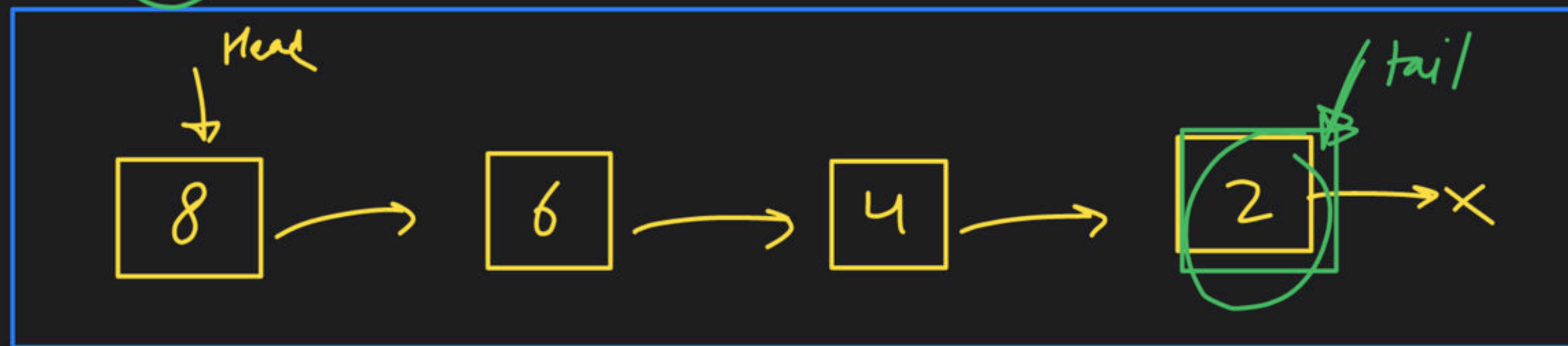
Special class

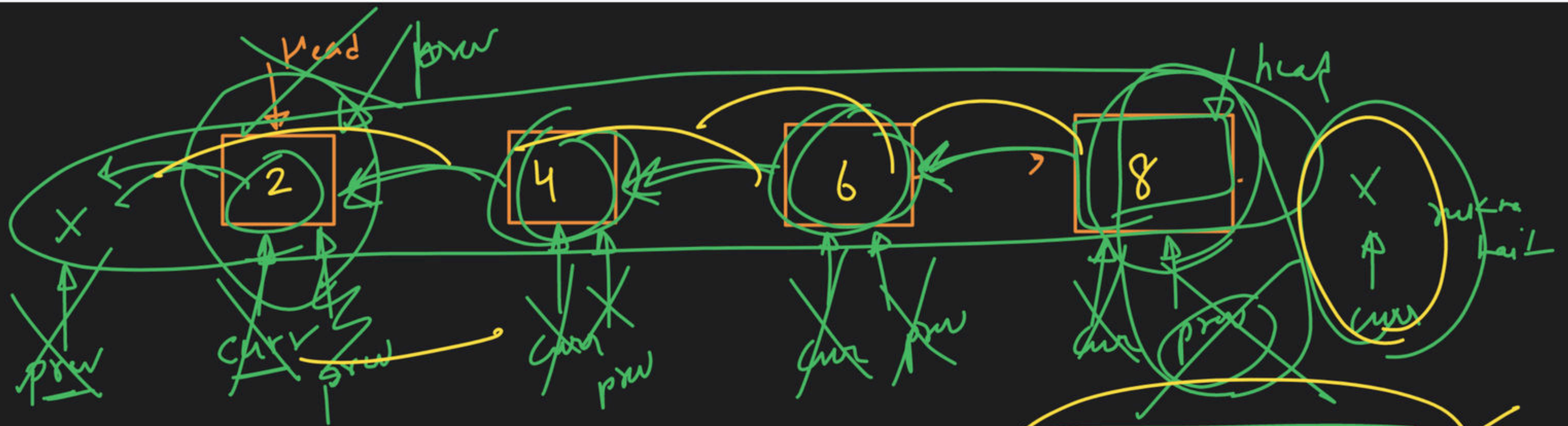
→ Reverse a Linked List

i/p →

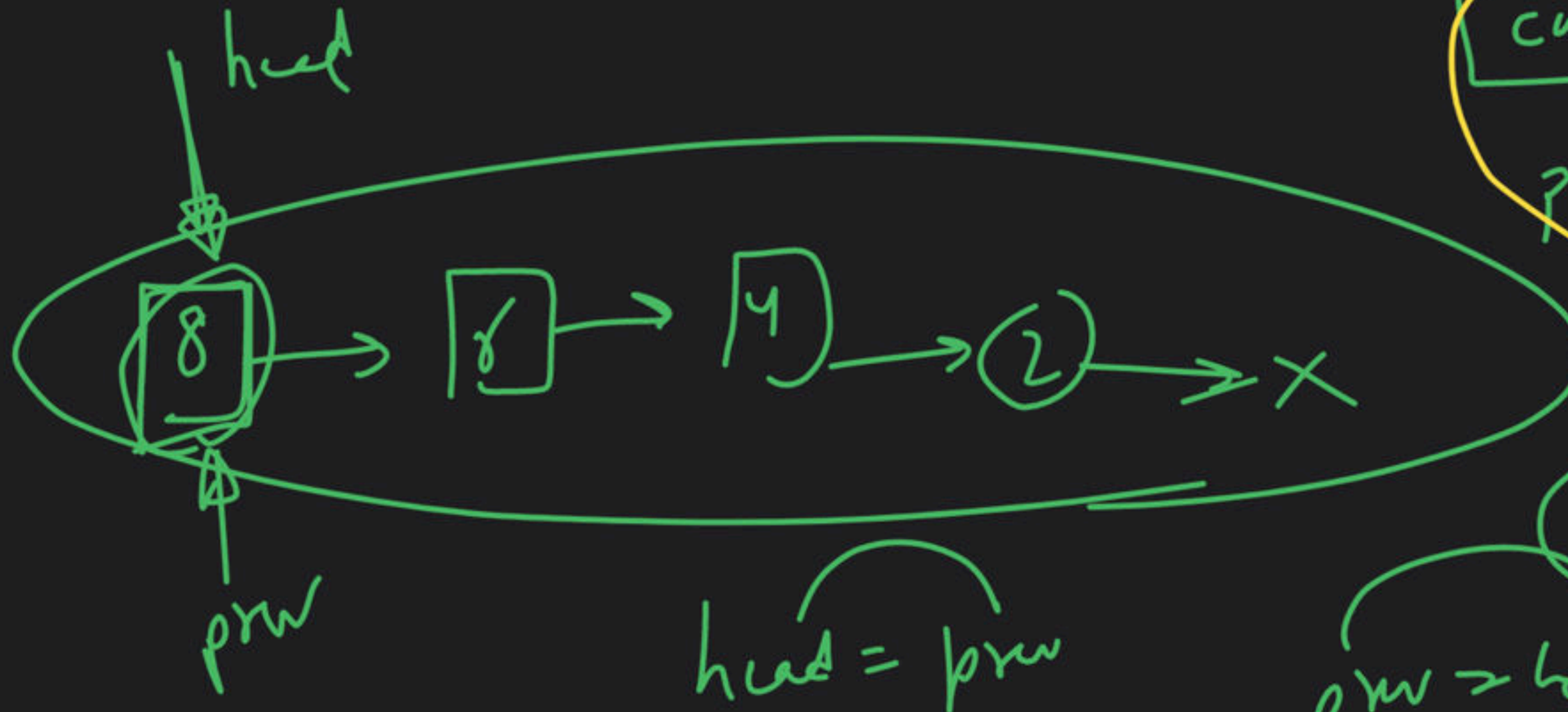


o/p →



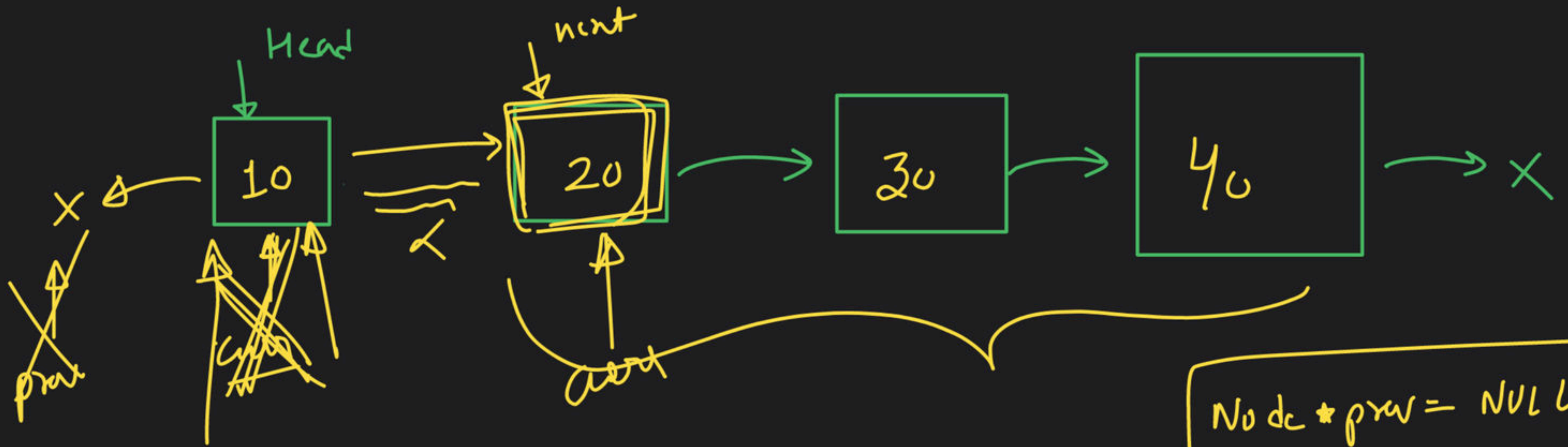


curr → next = prev
 prev/curr → null



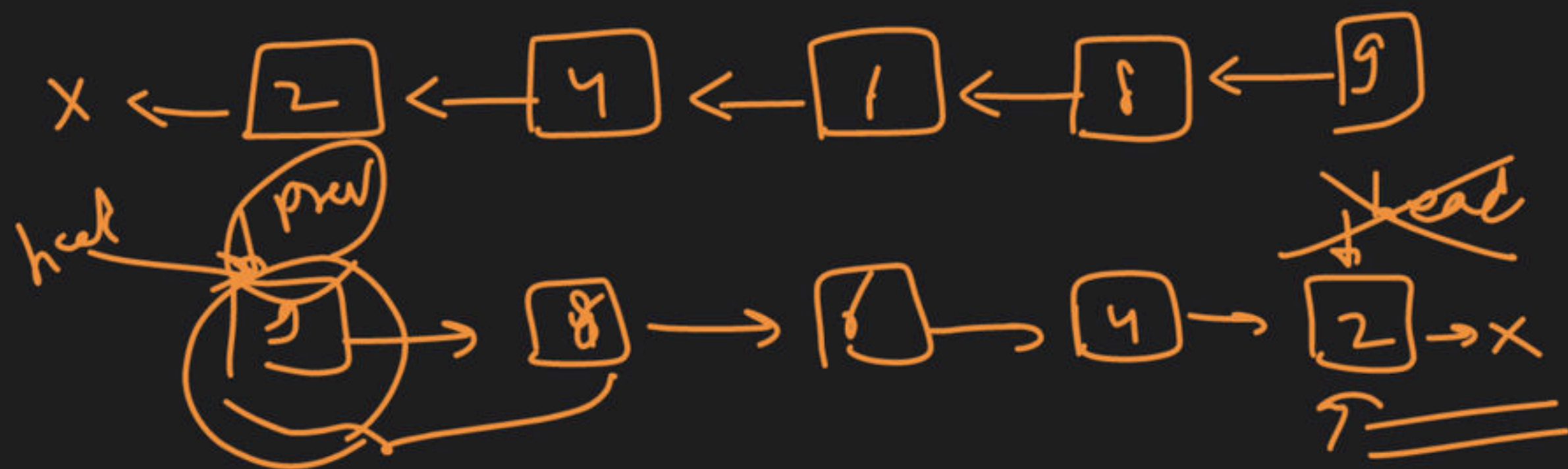
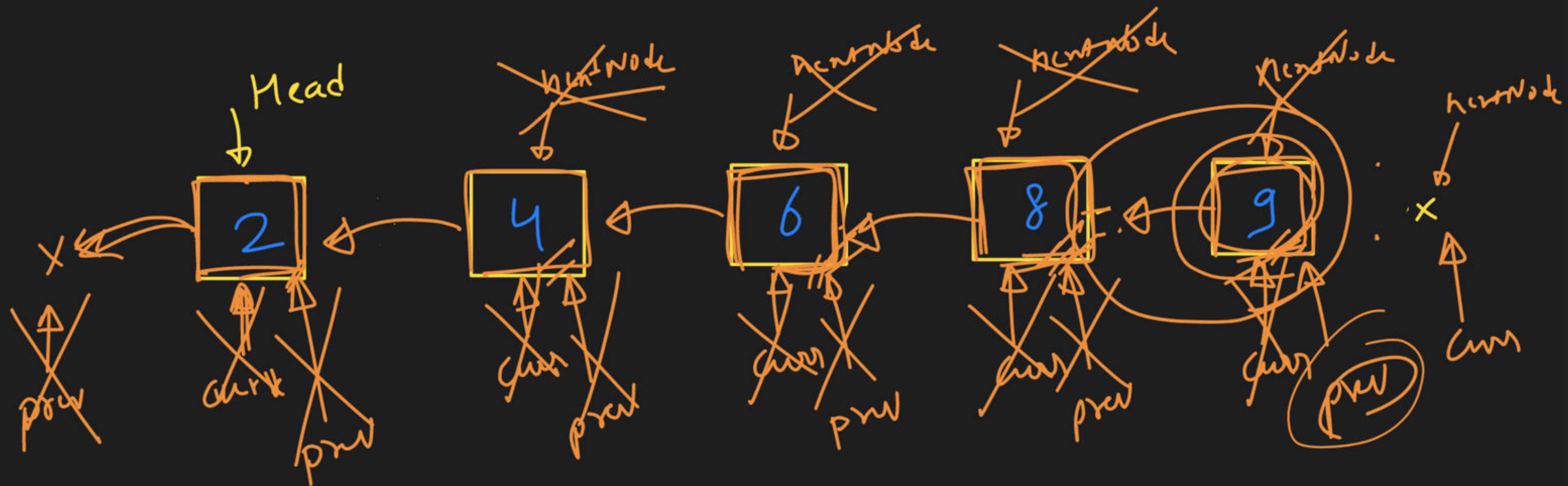
head = prev

head = prev
 prev = head



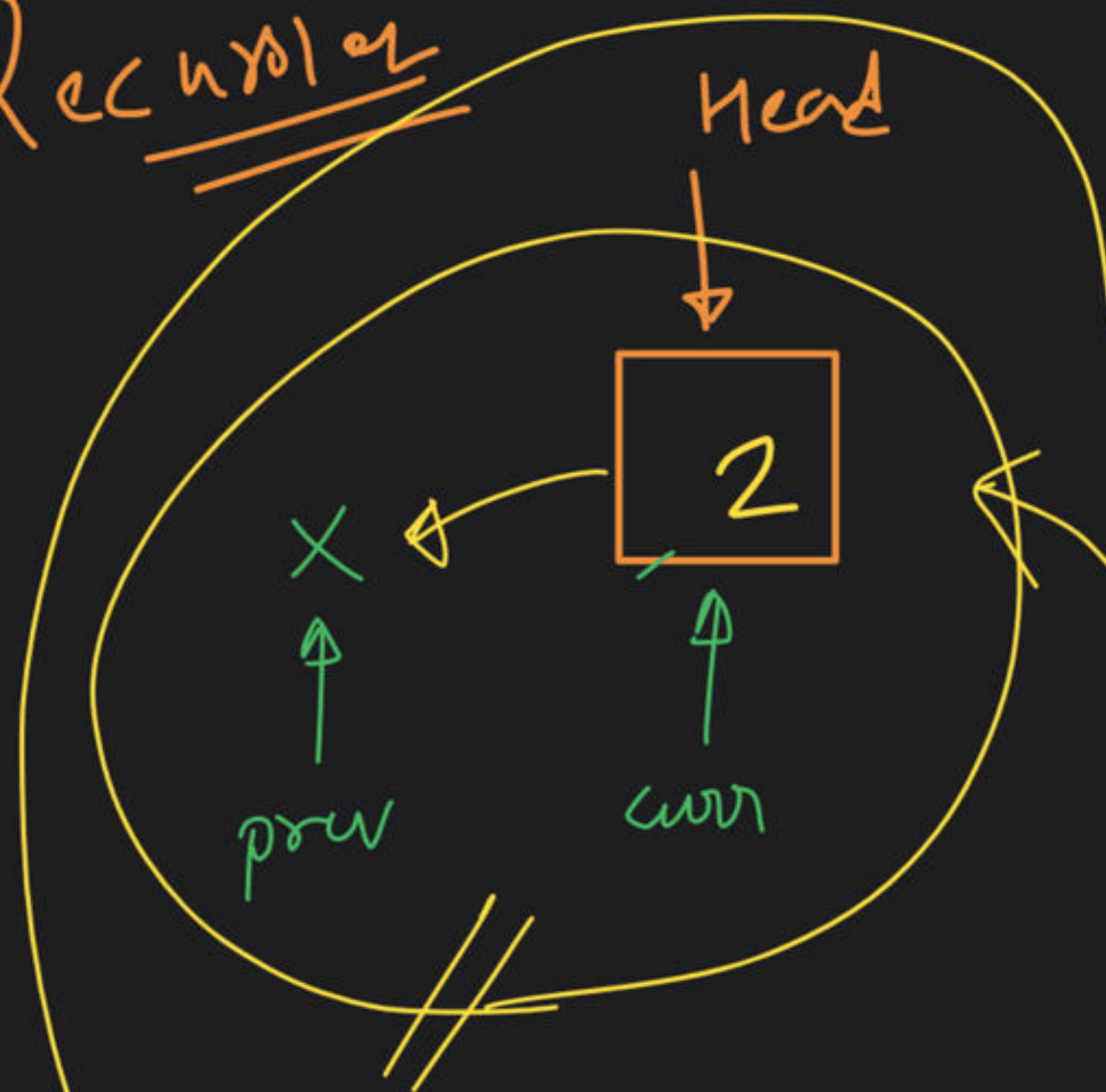
```
Node *prev = NULL;  
Node *curr = head;
```

```
while (curr != NULL)  
{  
    Node *next = curr->next;  
    curr->next = prev;  
    prev = curr;  
    curr = next;  
}  
head = prev;
```



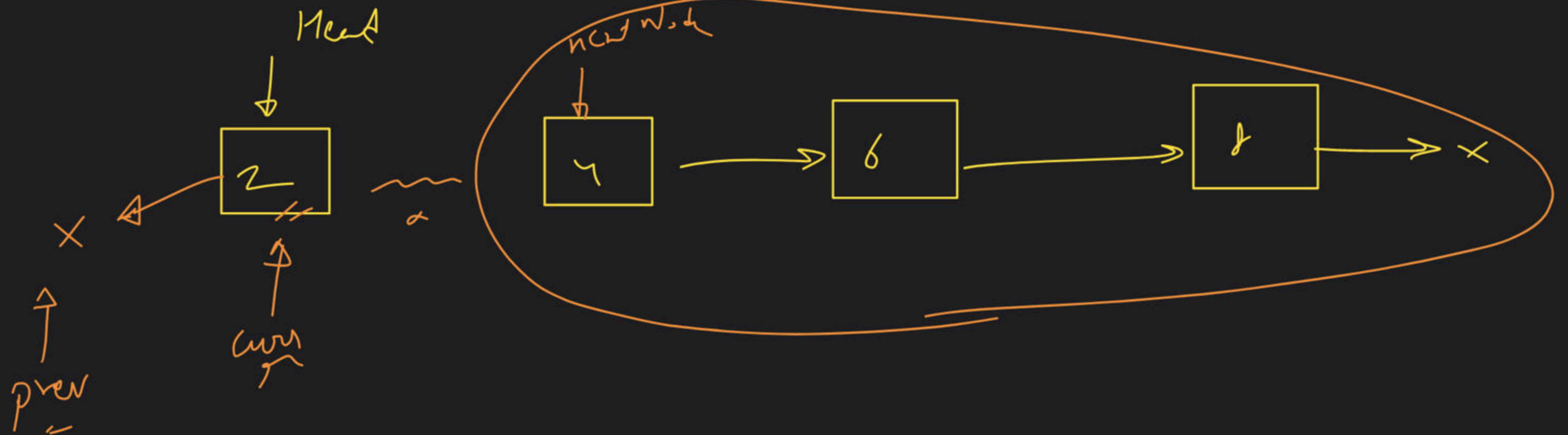
- key node
- (A) nextNode = curr->next
 - (B) curr->next = prev
 - (C) prev = curr
 - (D) curr = nextNode
- head = prev

Recursion



Recursion





Runke Kab Lei

```

if (curr == NULL)
{
    return prev
}

```

Node * prev = NULL;

Node * curr = head

Node * next = curr->next

curr->next = prev

Rec

Middle of a Linked List

$$n=5$$

$$\lfloor \frac{n}{2} \rfloor = \frac{5}{2} = 2+1$$

~~odd~~
5

Head

2

1

4

2

6

3

8

4

10

5

x

~~$\frac{n+1}{2}$~~ → mid node

Head

2

1

4

2

6

3

8

4

10

5

12

6

x

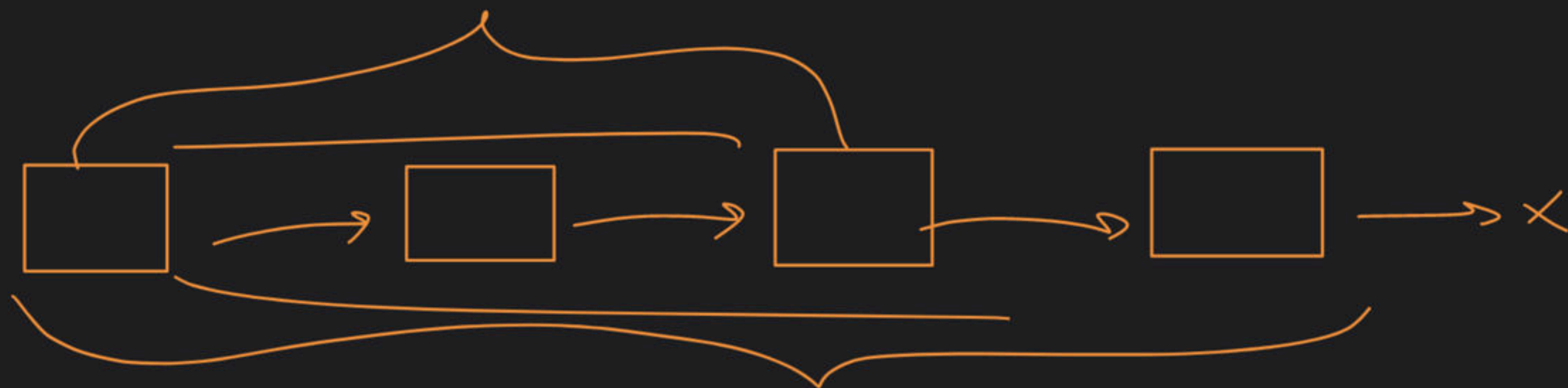
n=6

$$\frac{n}{2} = \frac{6}{2} = 3$$

$\frac{n}{2} + 1 \Rightarrow$ mid node

Algo → $\frac{n}{2} + 1 \rightarrow$ traverse

~~Even~~
6



~~2 pass~~ → len → $O(N)$
→ mid → $O(\frac{N}{2})$

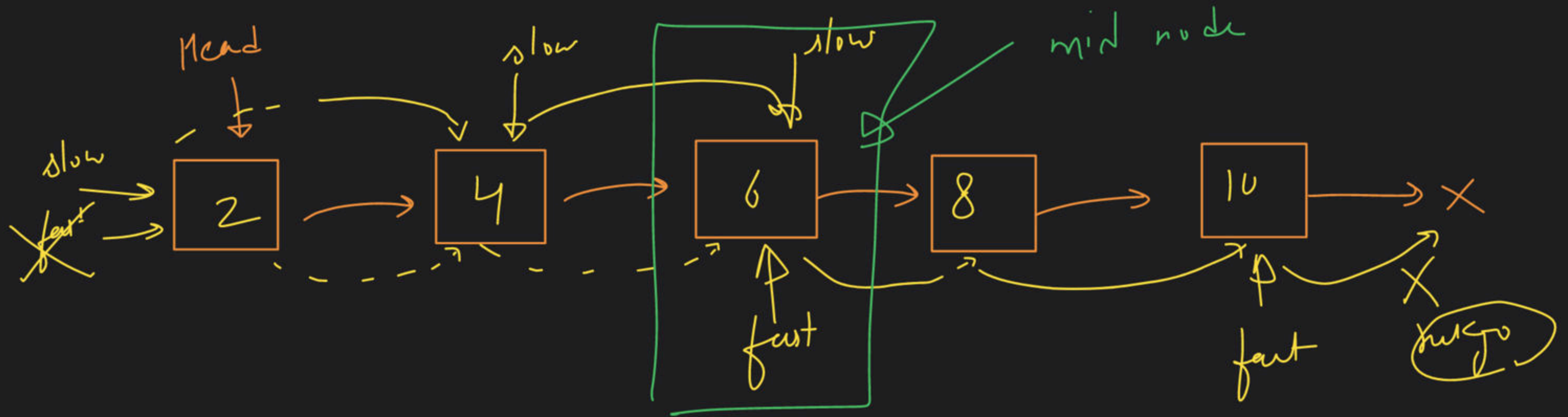
$$O(N) + O(\frac{N}{2}) \rightarrow O(N)$$

Single Pass

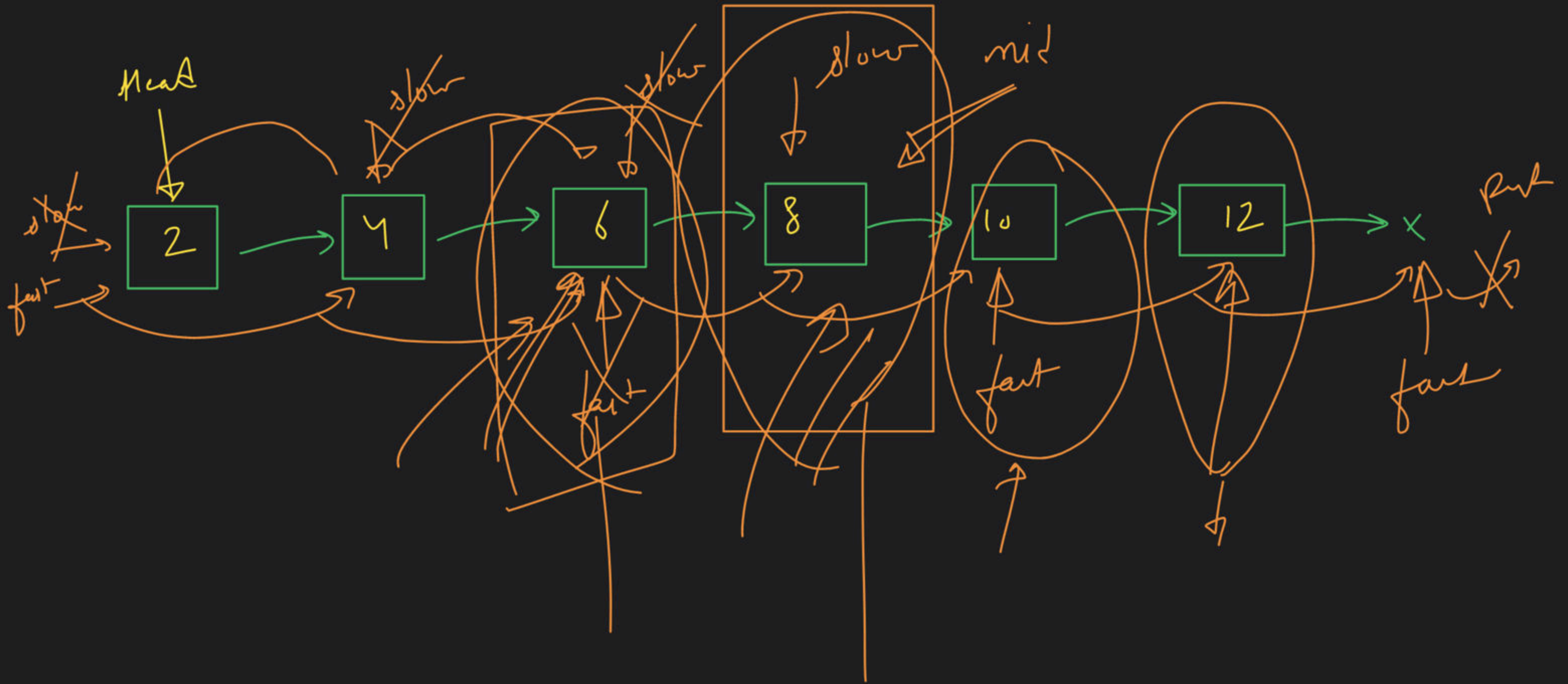
Slow & fast

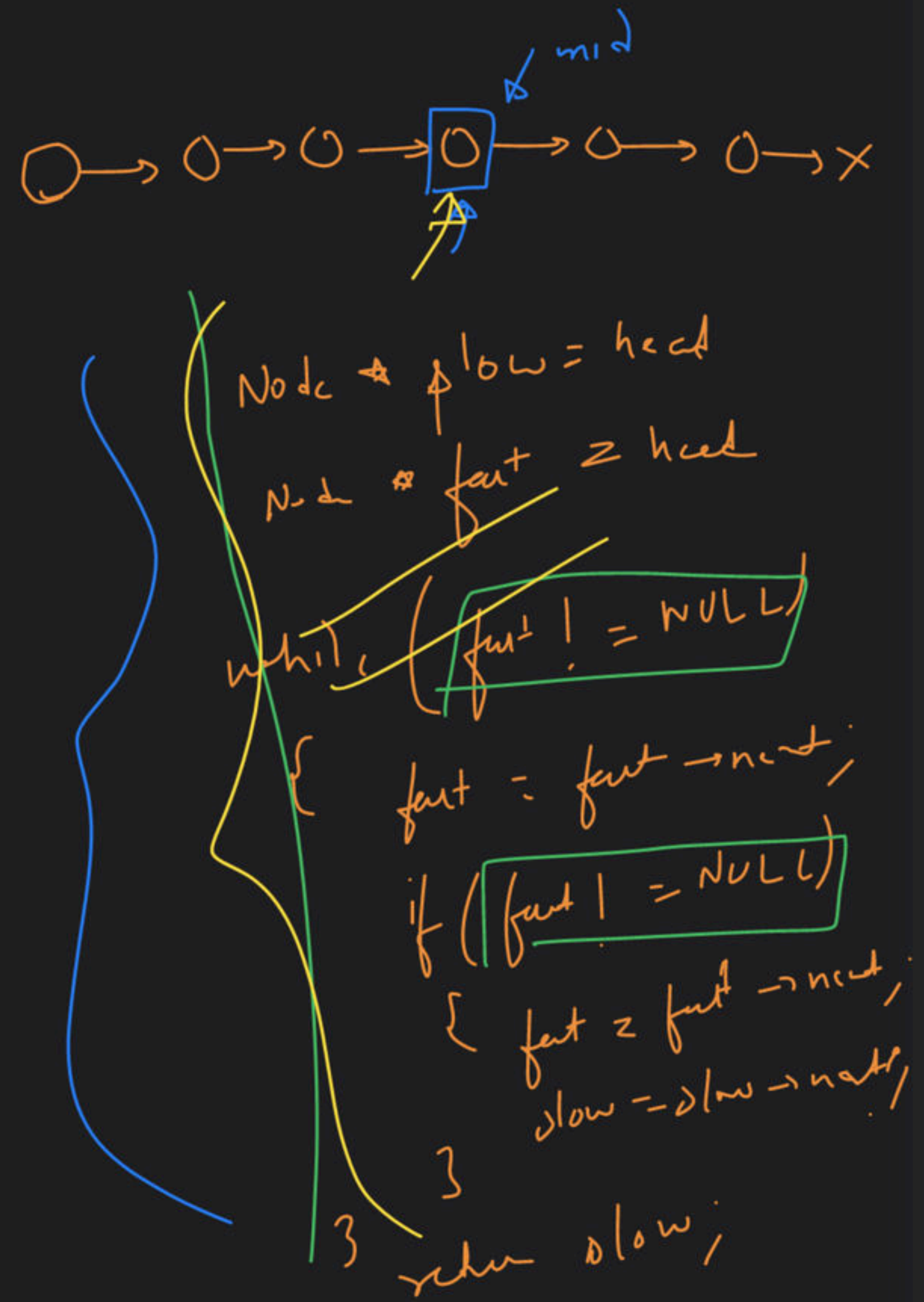
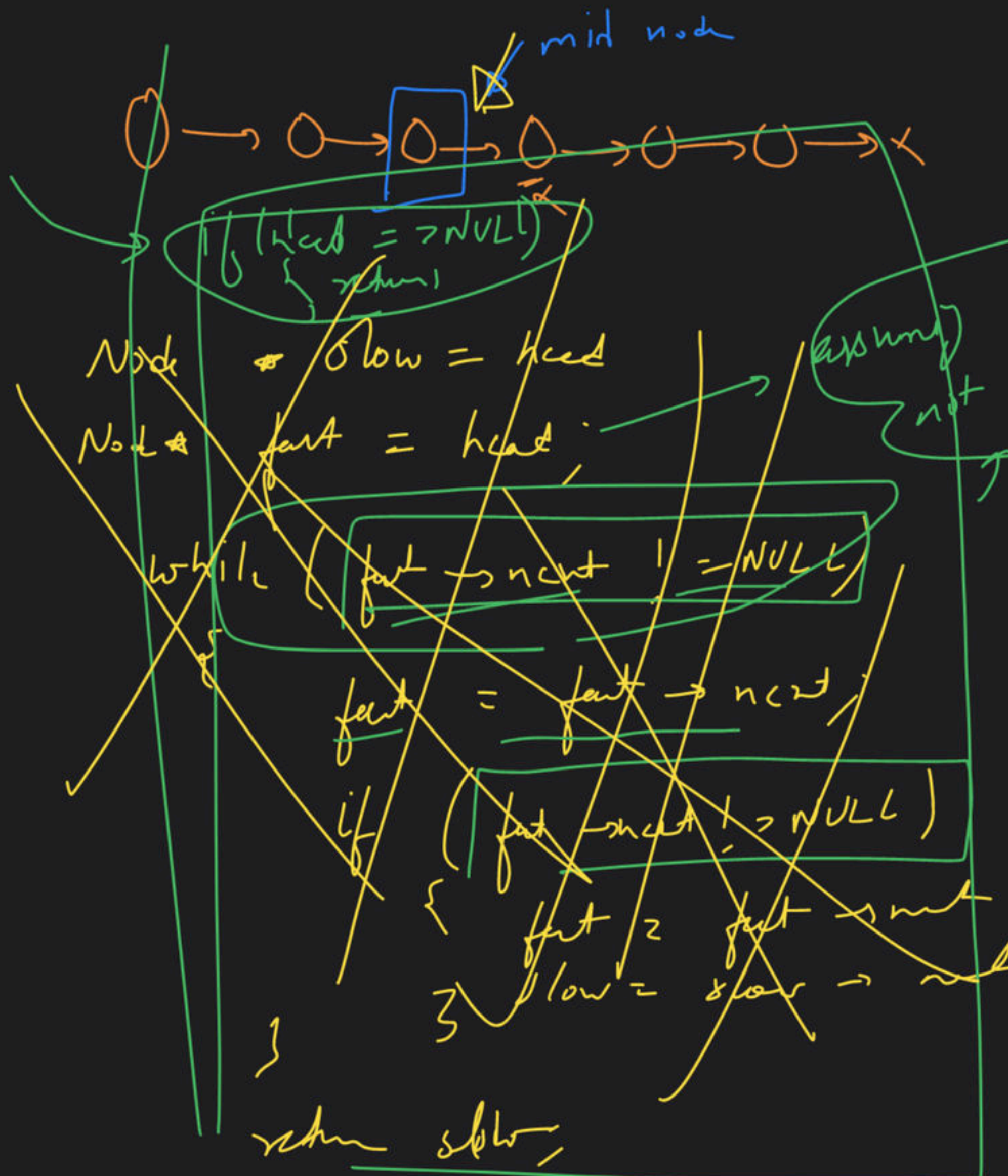
Slow $\rightarrow$ 1 step

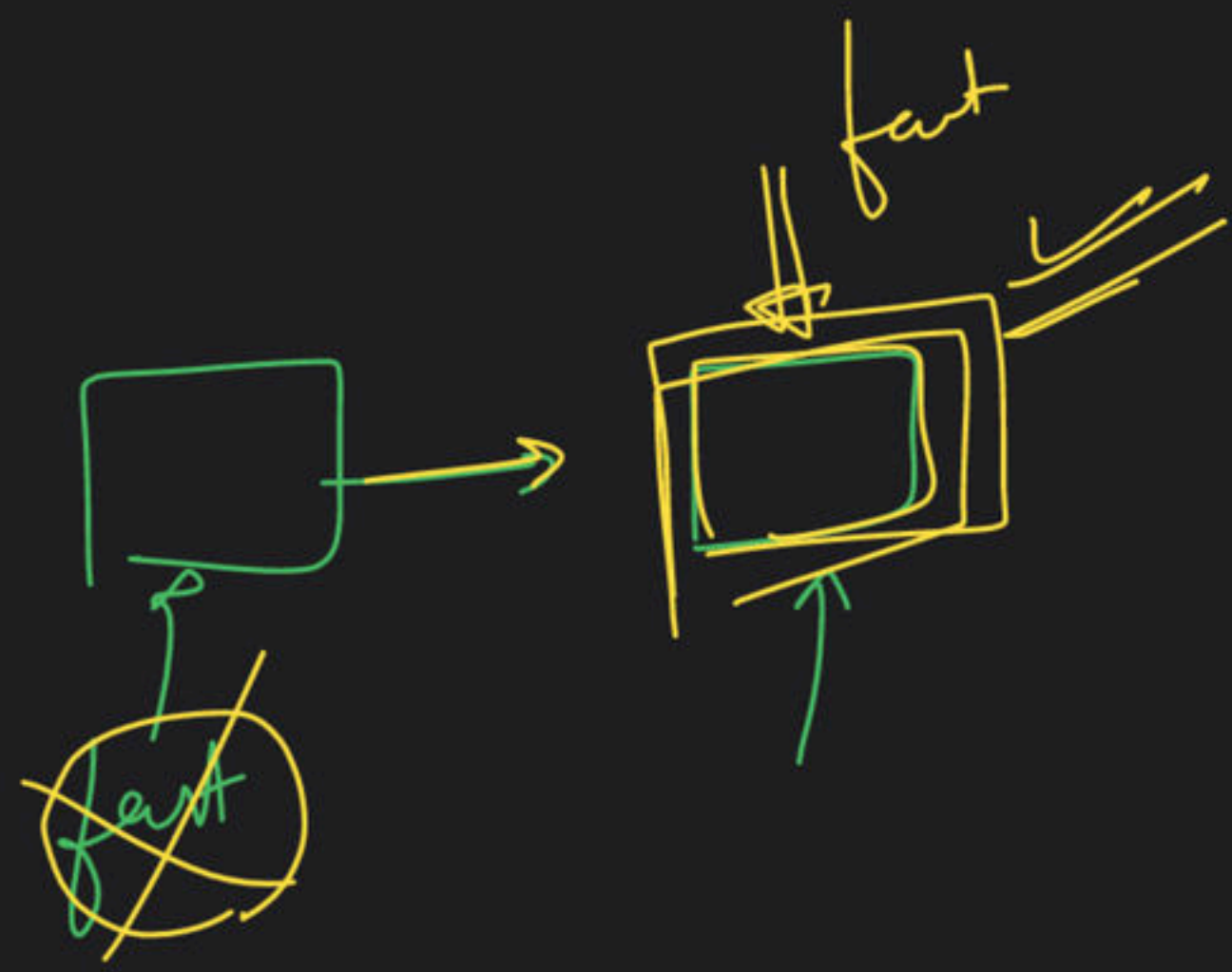
fast $\rightarrow$ 2 step

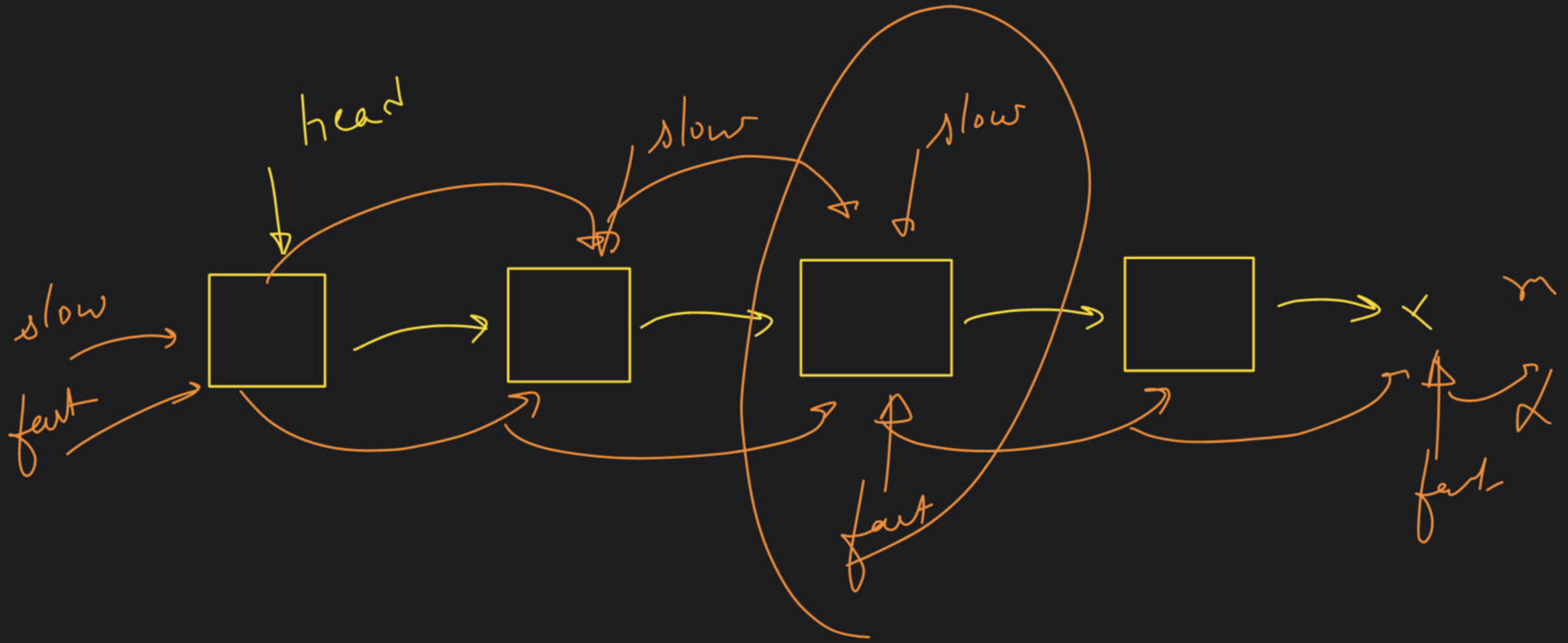


(A) slow/fast $\rightarrow$ init









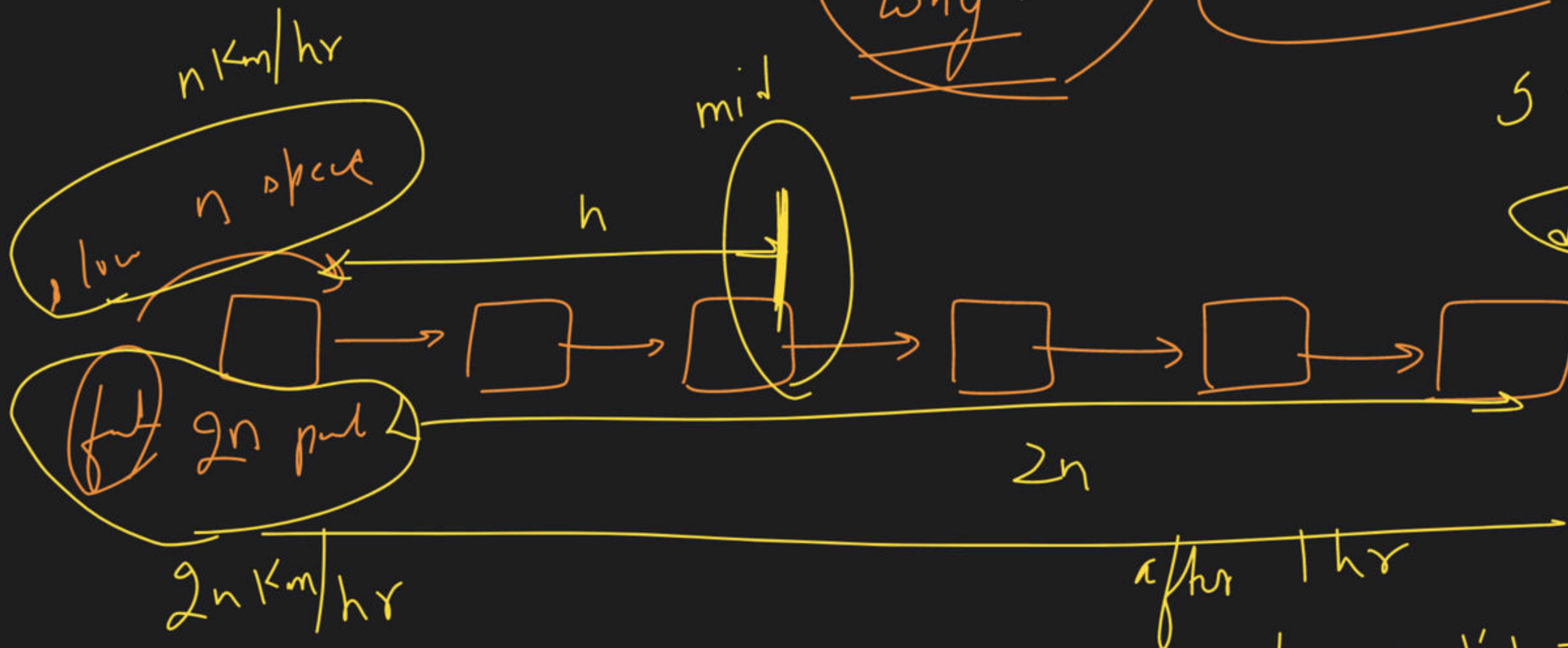
what

why?

ratio of

$$s = \frac{d}{t}$$

$$d = s \times t$$



after 1 hr

$$\text{slow} \rightarrow d_{\text{slow}} = n \times 1 = n \text{ km}$$

$$\text{fast} \rightarrow d_{\text{fast}} = 2n \times 1 = 2n \text{ km}$$

after 5 hr

$$\text{slow} \rightarrow \text{dist} = 10 \times 5 = 50 \text{ km}$$

$$\text{fast} \rightarrow 20 \times 5 = 100 \text{ km}$$

slow $\Rightarrow 10 \text{ km/hr}$

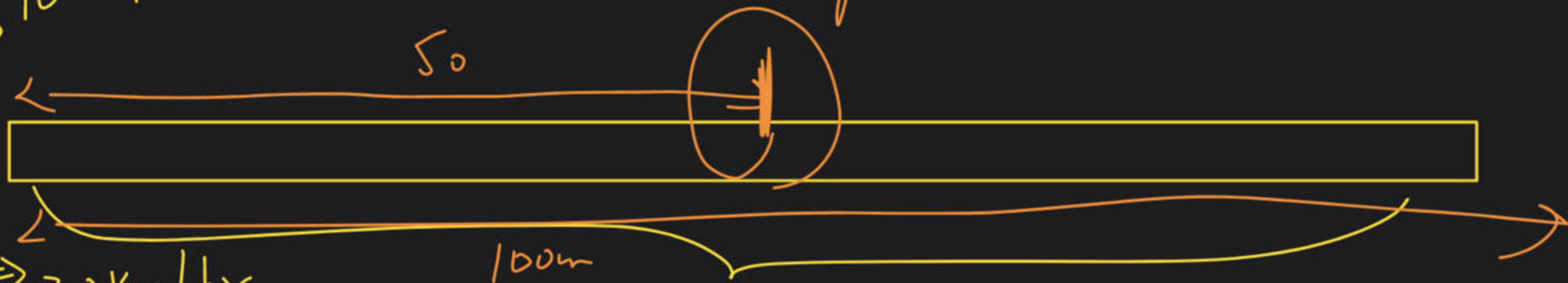
mid point

50

fast $\Rightarrow 20 \text{ km/hr}$

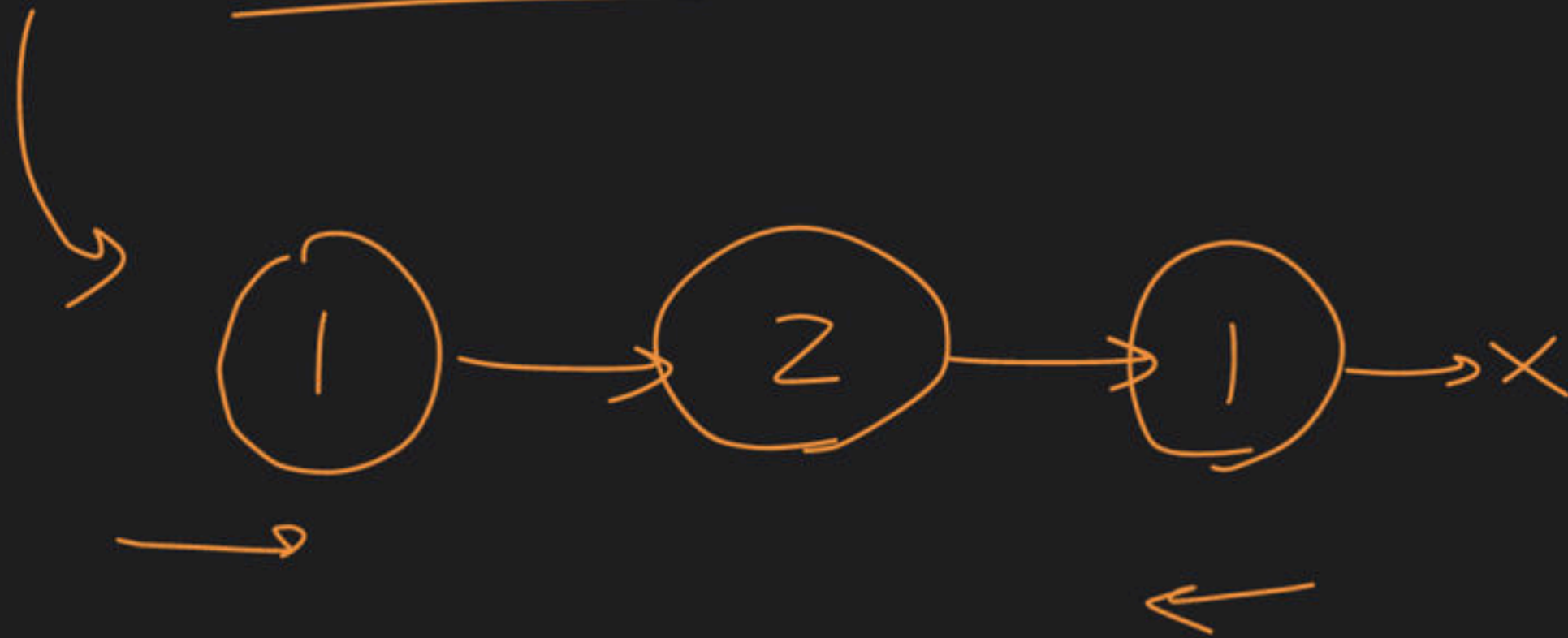
100m

100km

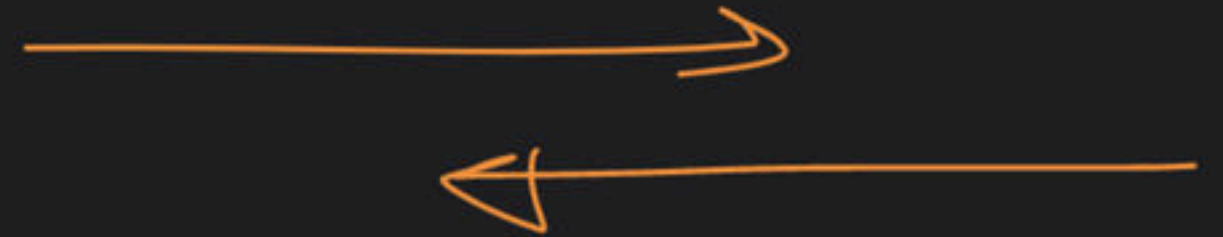


③

Palindrome



babbar



RACECAR

NITIN

#1

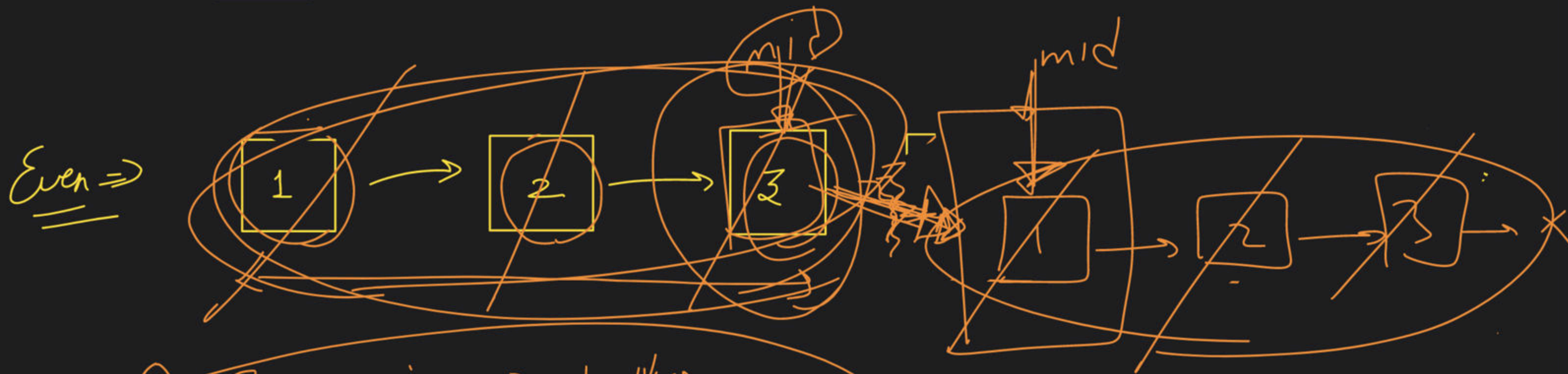
~~gewisse Linkel mit~~

Compare $\rightarrow$ True

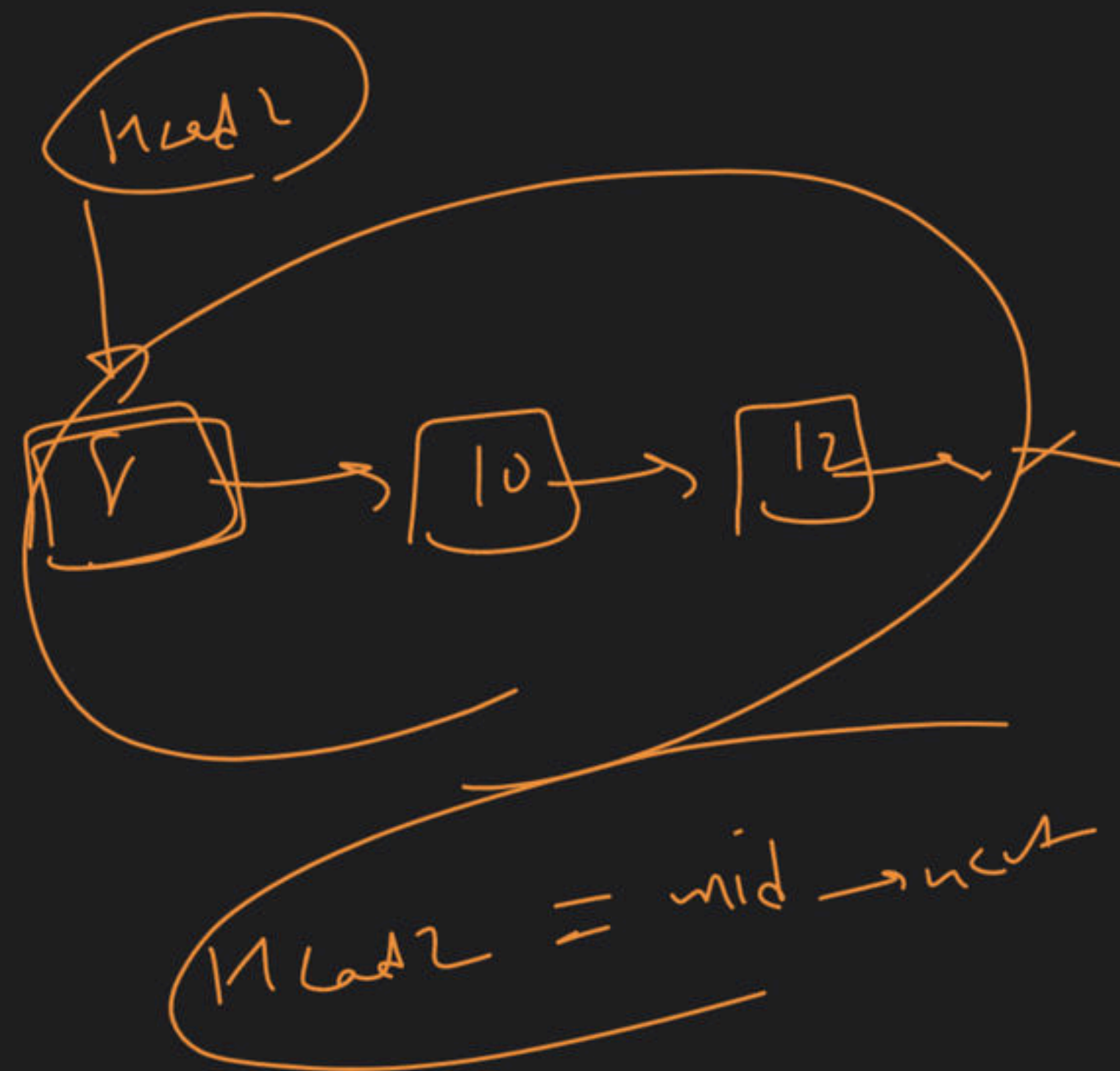
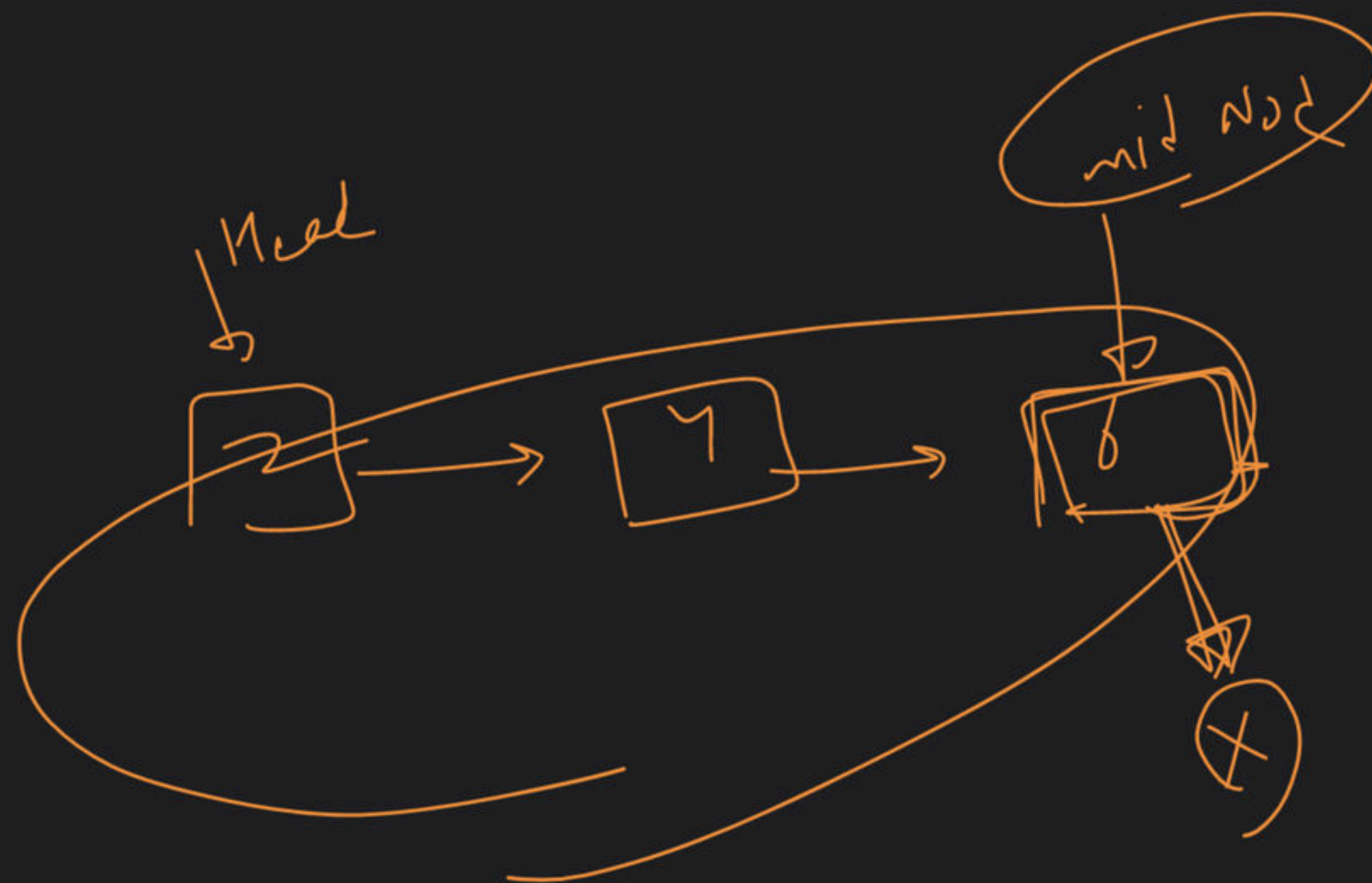
'don not' $\rightarrow$ false

$$S \rightarrow O(N)$$

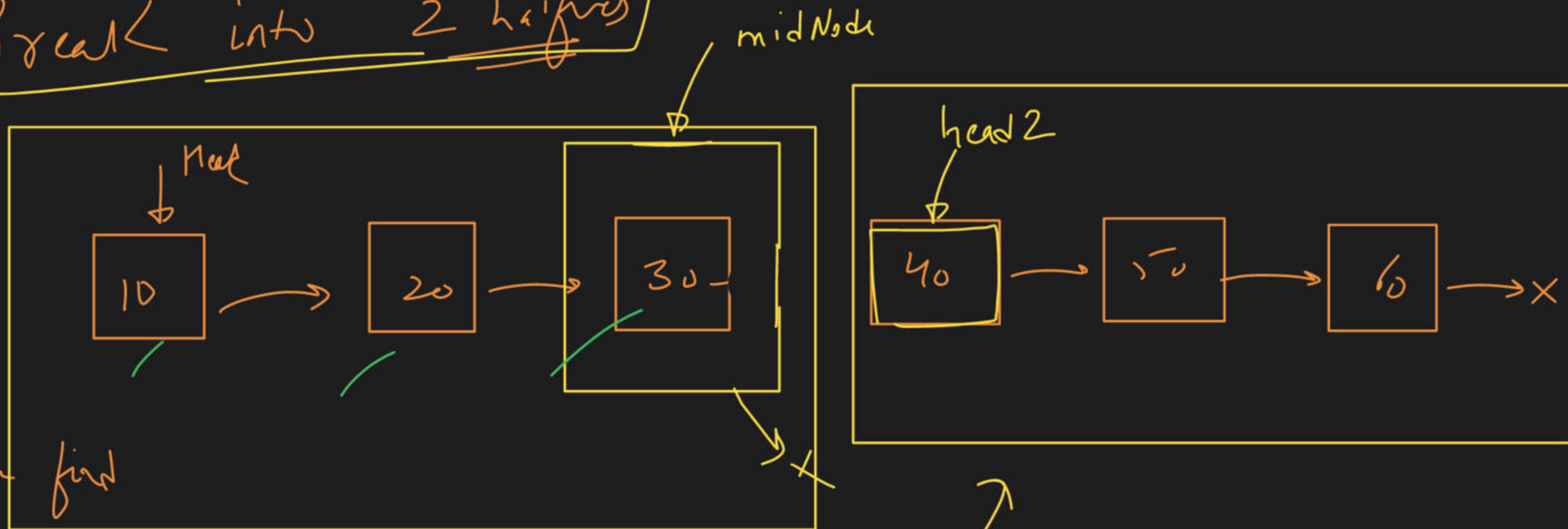

(2 min)



- (A) Break into 2 halves
- (B) Second list $\rightarrow$ reverse
- (C) Compare both Linked List



(A) Break into 2 halves

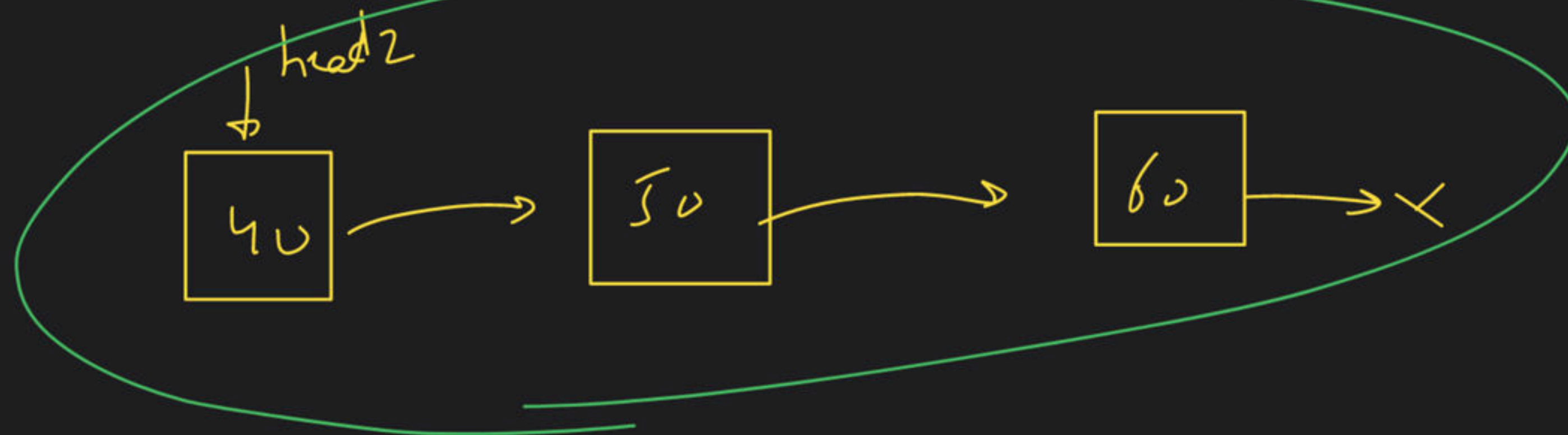


① midNode find

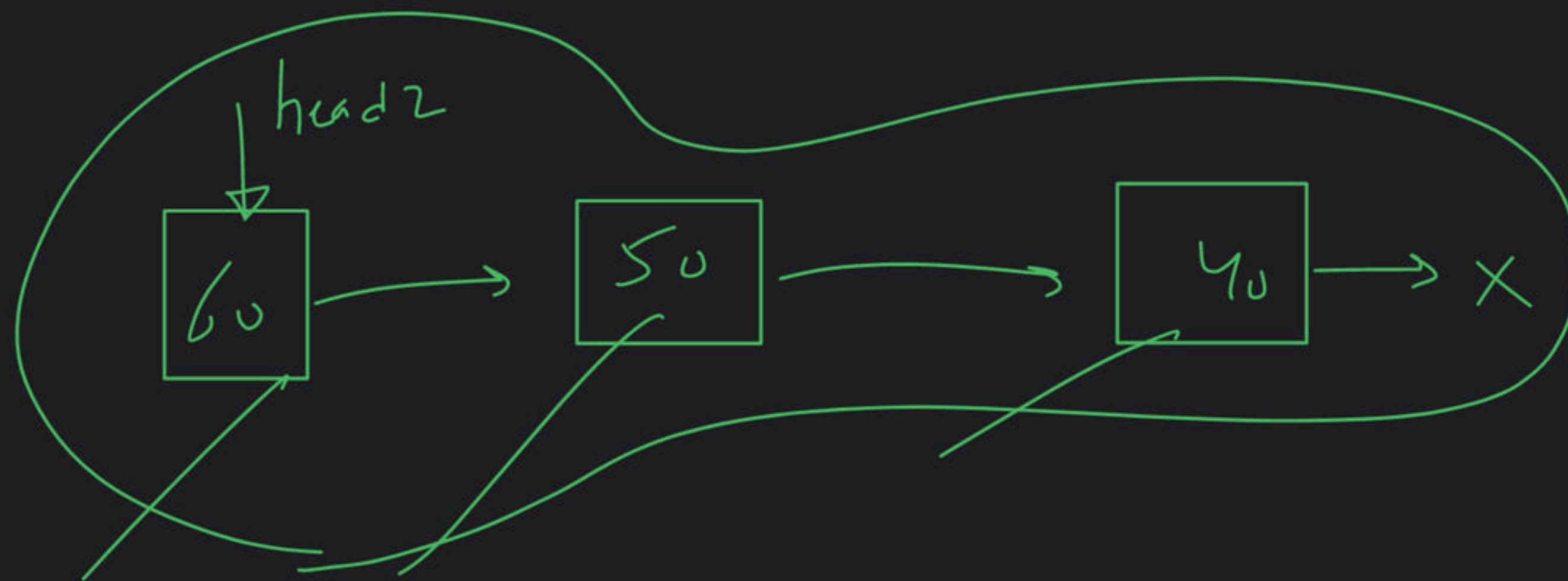
② head 2 set

③ midNode → next = NULL

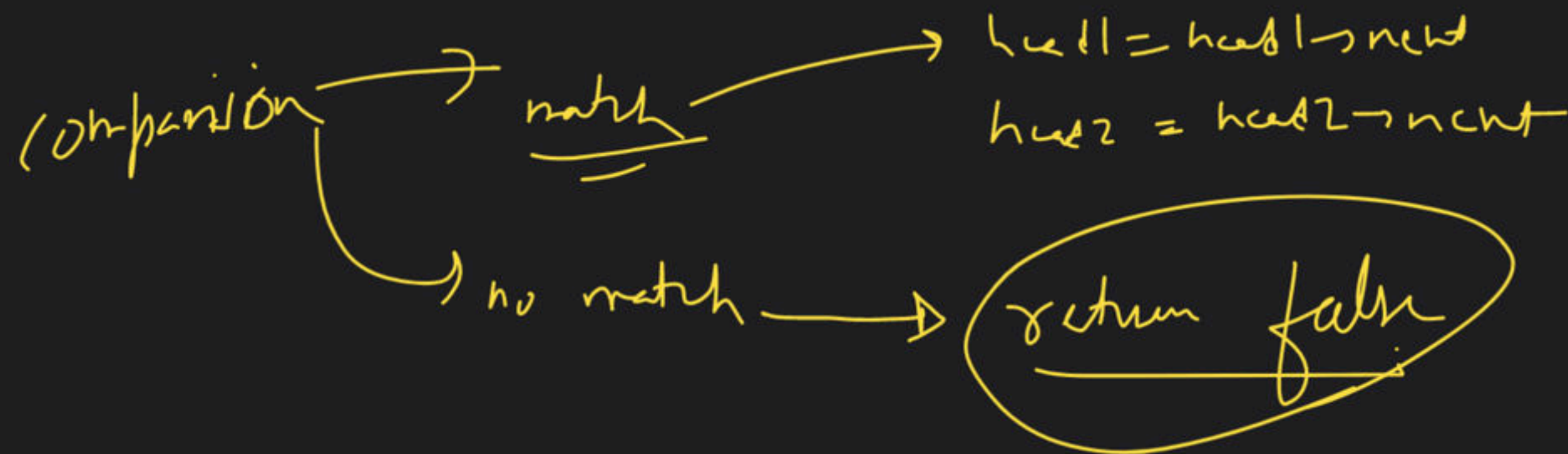
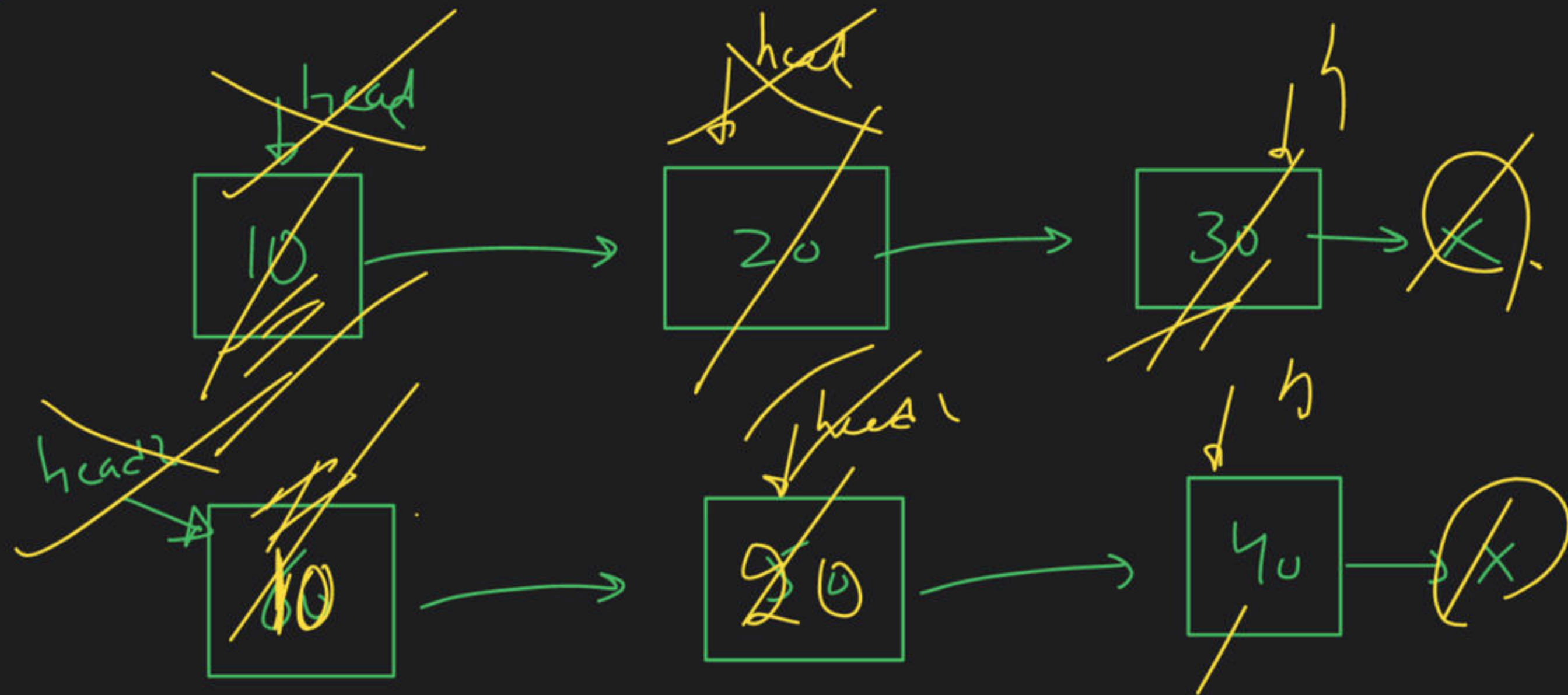
(B) Reverse Second half



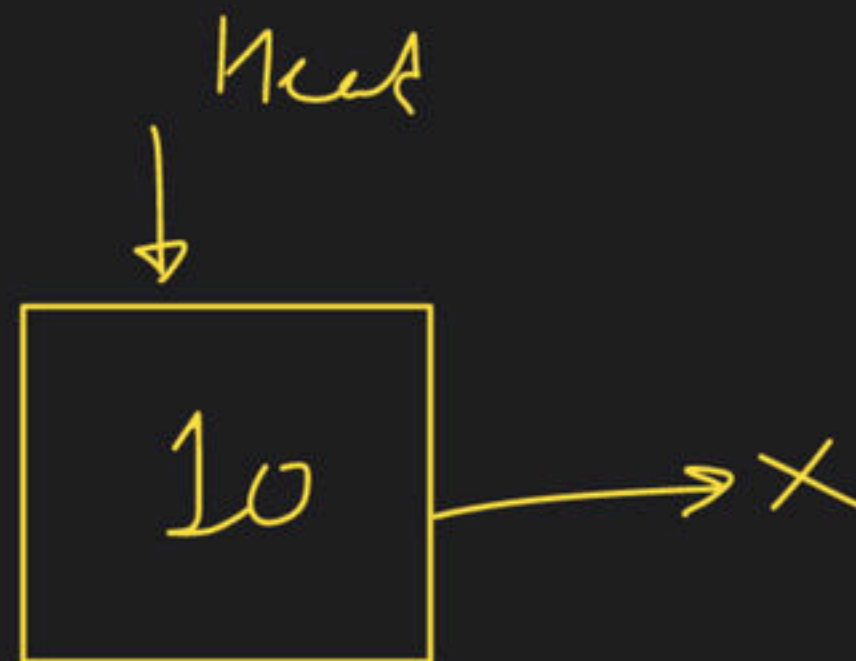
Reverse



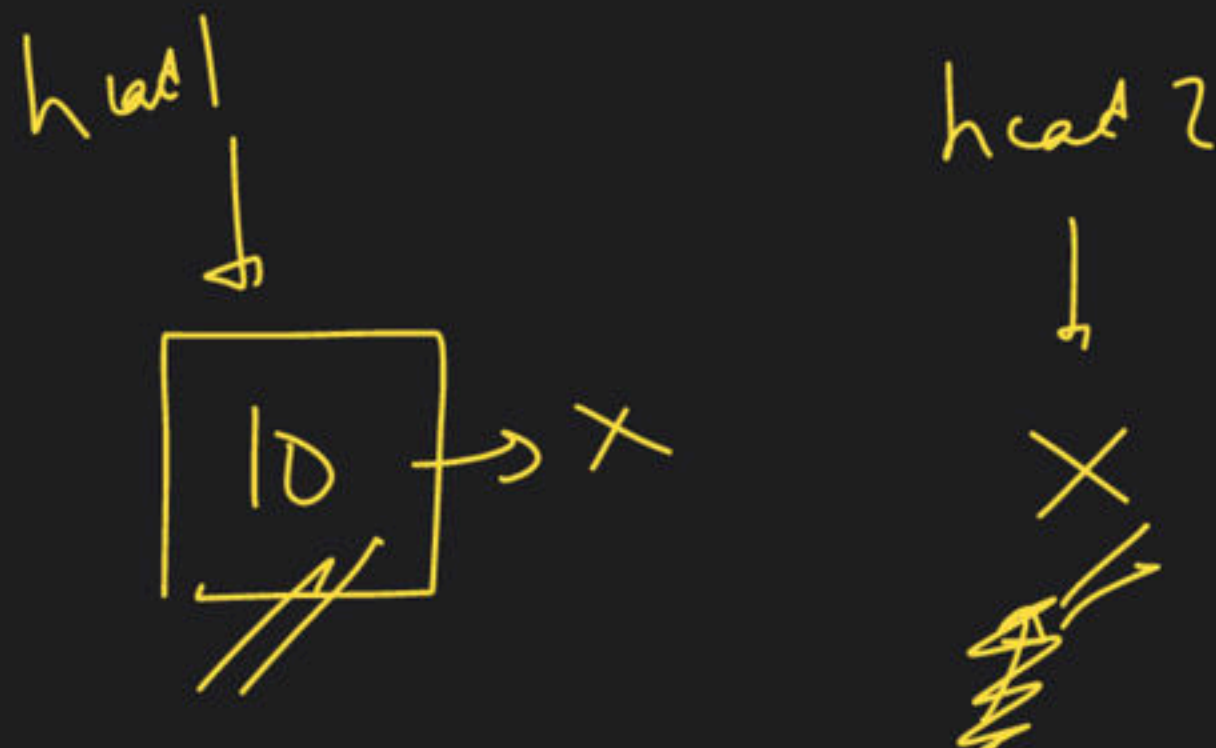
① Compare both List



(A)

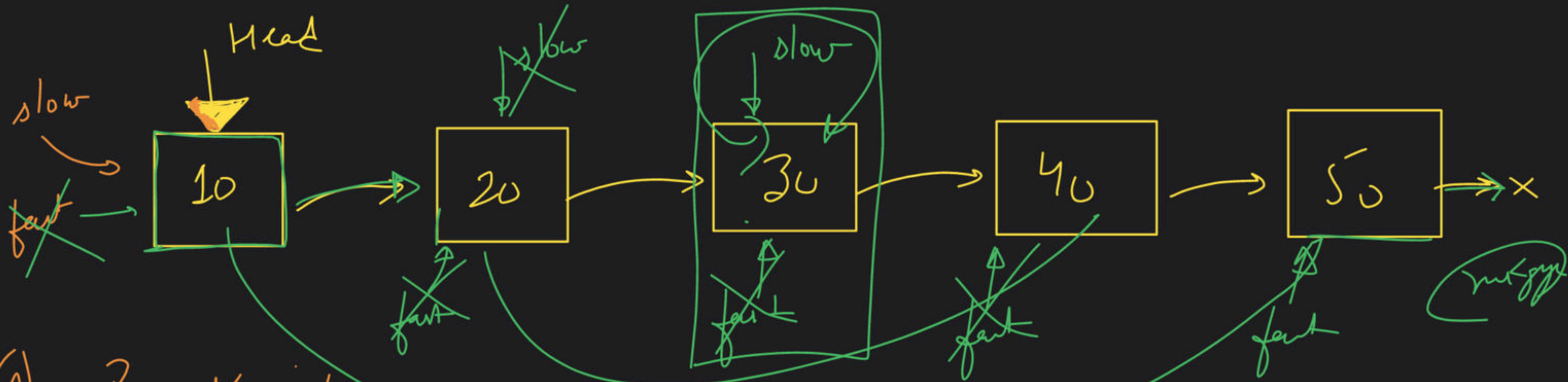


(B)

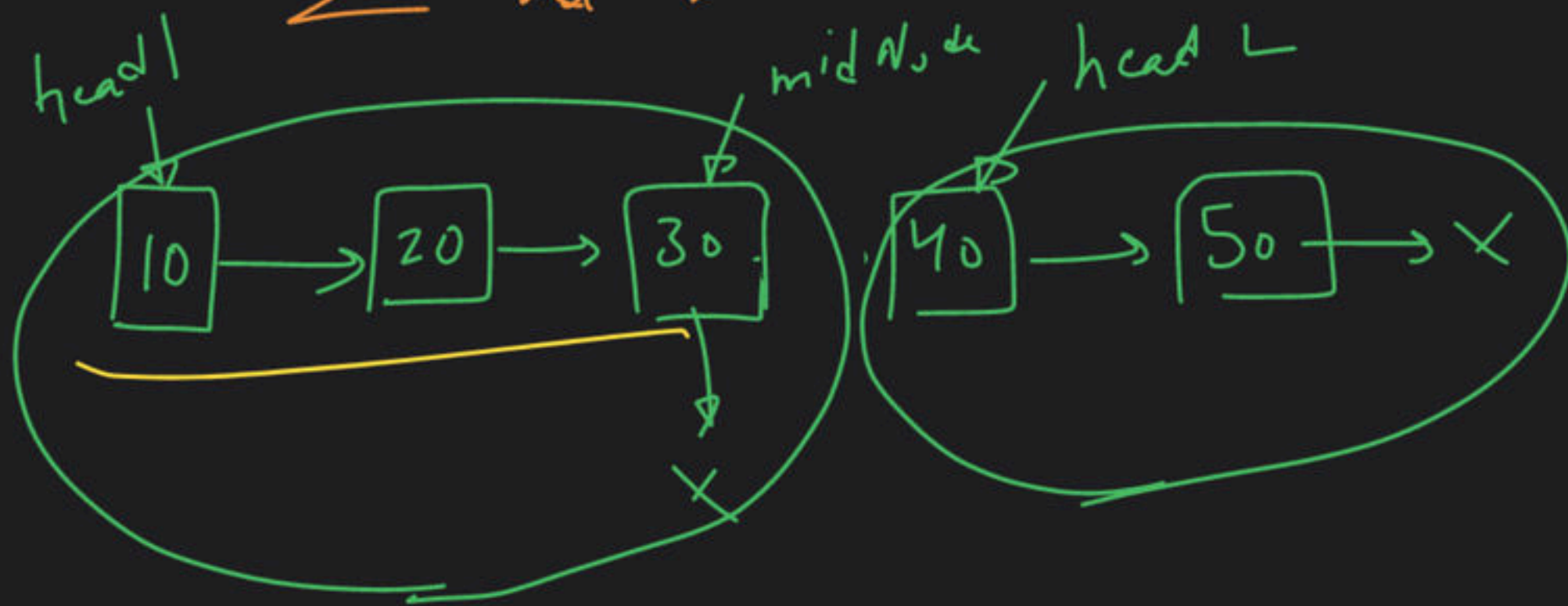


(C)

return true →



(A) Break into
2 halves

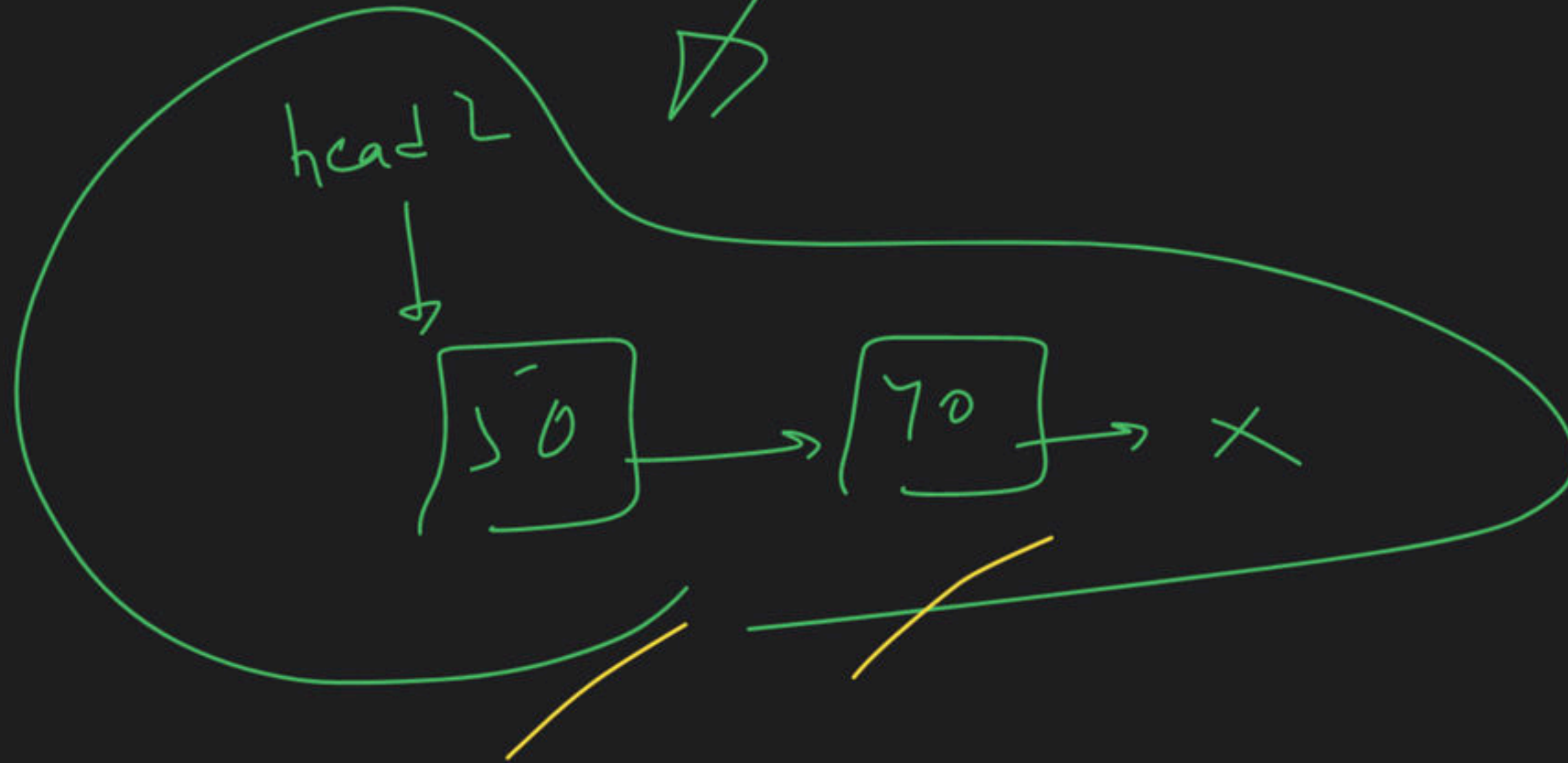


```

while (fast != NULL)
{
    fast = fast->next;
    if (fast != NULL)
    {
        fast = fast->next;
        slow = slow->next;
    }
}

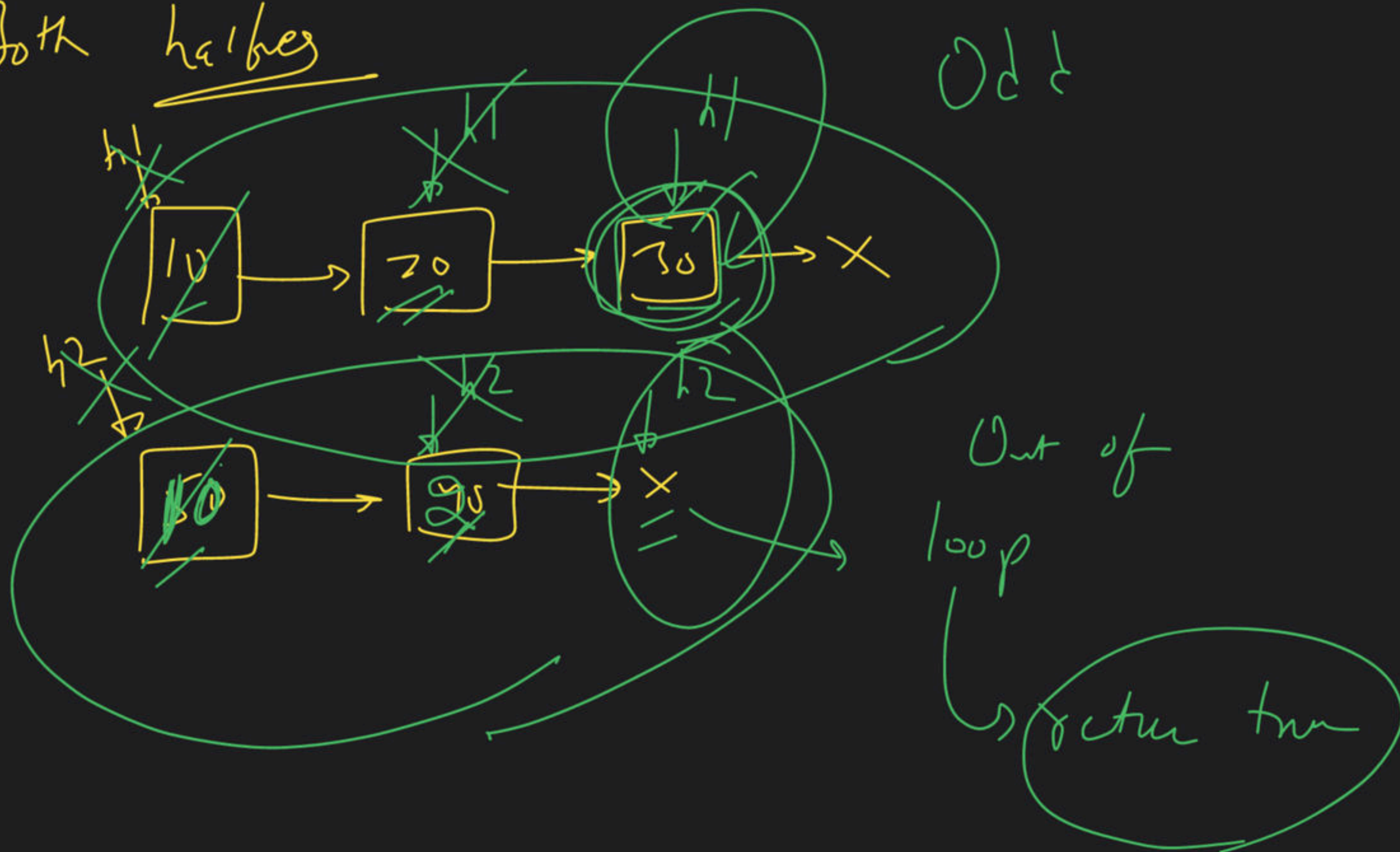
```

③ ~~Reverse~~ Reverse 2nd half



→ Compare Both halves

match →
h1 → next
h2 → next
if match
return false



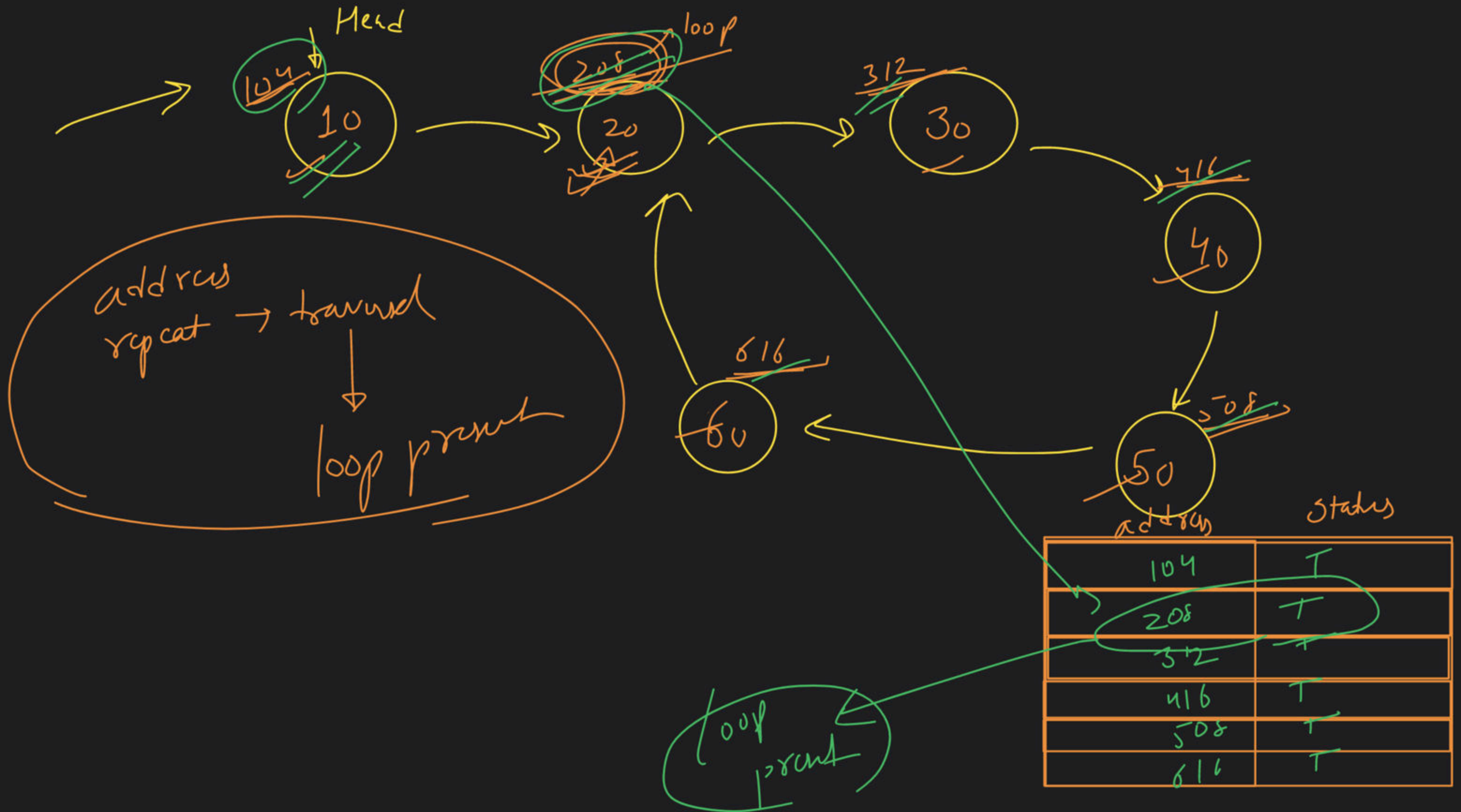
(A) Break $\rightarrow$ mid $\rightarrow O(N)$

(B) Run $\rightarrow O(N)$

(C) Low $\rightarrow O(N)$

$O(N)$

→ Check cycle in a Linked List



dot 2.0

